MANIPAL UNIVERSITY
JAIPUR

PROJECT REPORT

AceOS

Asset, Compliance & Equipment
Operations System

Object Oriented Programming

Submitted by

Smit Hitnalli

Registration No: 2502051938
Section: N3

Manipal University Jaipur
B.Tech Computer Science and Engineering

Abstract and report guide

AceOS provides a traceable workflow for institutional equipment operations. Its repair process separates technician completion from administrator verification and retains every correction as a new submission. Complaint reasons, repair reviews, audit events and safety evidence make the outcome explainable. Relational constraints and guarded service operations work together to preserve consistency. The implementation also demonstrates encapsulated lifecycle methods, JavaFX inheritance, JDBC interface polymorphism, generics and transaction resource management. The result is a working local college project whose documented verification includes 89 configured checks with no failures, errors or skips in the final recorded run. This report uses implementation evidence rather than claiming new experiments were performed during report preparation.

Report structure

This submission focuses on Object Oriented Programming. The following page guide includes the cover and references within the required 20 pages.

Page	Topic
3	Project background and objectives
4	Use cases and object responsibilities
5	Layered object-oriented architecture
6	Class relationships and collaboration
7	Encapsulation and valid state changes
8	Constructors and method overloading
9	Abstraction and interface contracts
10	Inheritance in the JavaFX interface
11	Polymorphism and method overriding
12	Associations, records and collection safety
13	Enums, generics and type safety
14	Exceptions and resource management
15	Repair lifecycle as object behaviour
16	Authentication and guarded service actions
17	JavaFX interaction and background work
18	Verification and implementation results
19	Limitations, future scope and conclusion
20	References and source evidence

Project background and objectives

AceOS stands for Asset, Compliance & Equipment Operations System. It is a local JavaFX application for managing institutional assets, complaints, repairs, inspections and preventive schedules. The same implemented application supports two academic perspectives: relational data management and object-oriented software construction.

Campus equipment maintenance involves several linked facts: which asset has a problem, who reported it, which technician was assigned, what work was submitted and which administrator accepted the result. A collection of disconnected spreadsheets makes these links difficult to validate and can lose the history of a correction.

AceOS connects those records through defined identities and workflows. Requesters report faults and track their own complaints. Technicians execute assigned work. Administrators review complaints, coordinate assignments and verify completed repairs. Inspection failures can also restrict an asset independently of its repair history.

Implemented scope

The report documents the current v2.0.0 application and schema migrations V001-V005. It does not describe a proposed online campus portal. The application runs locally using Java 24, JavaFX 24.0.2, JDBC, MySQL 8 and Maven. A separately configured database and private credentials are required.

Use cases and object responsibilities

Role	Responsibility	Key restriction
Requester	Report, comment, withdraw, reopen	Own complaints; lifecycle rules apply
Technician	Repair, correct and inspect	Assigned work and permitted history
Administrator	Review, assign, verify, manage	Required reasons and evidence checks

The main use cases are reporting an asset problem, reviewing a complaint, assigning a repair, recording completion, returning incomplete work, submitting corrections and accepting the final result. Related use cases manage inspections, safety holds, preventive schedules and accounts.

Behaviour belongs with its responsibility

Complaint and WorkOrder represent lifecycle behaviour. AceOsService coordinates authorized application actions. Repositories map durable rows to objects. JavaFX views collect input and display results. This division keeps interface event handlers from becoming the owner of database transactions or business authorization.

An end-to-end example

When a technician submits a correction, the view supplies the work and displayed evidence identifiers. The service validates the current session, assignment and state. The domain transition represents RETURNED to COMPLETED, while the repository inserts a new immutable submission and advances the persistent pointer. The transaction commits the action and audit together.

The three roles are values of UserRole. The implementation does not create Requester, Technician or Administrator subclasses of User. A role change therefore updates policy-relevant identity state rather than replacing the user with a new subtype.

Layered object-oriented architecture

Package / class	Responsibility
model	Entities, enums and bounded record projections
service / AceOsService	Public use cases and transaction coordination
repository	SQL and object reconstruction
database / Transactions	Connection ownership and rollback
security	Authentication, sessions and operation scope
ui / view	JavaFX layouts and input handling
ui / FxTasks	Background operation and UI callbacks

Views depend on application actions rather than embedding SQL. The service uses repositories as collaborators. Models express selected state rules without depending on JavaFX. JDBC types isolate most persistence code from concrete driver classes.

Cohesion and coupling

The separation makes each layer easier to inspect, but it does not imply a complete dependency-injection framework or a repository interface for every class. AceOsService directly constructs several repository collaborators. The design should be described as implemented, including that coupling.

Object lifetime

A service retains its active Session. Transaction instances are short-lived resources created for one operation. Views persist while users navigate their workspace. Reconstructed model objects represent a snapshot; persistent expected-state checks remain necessary when that snapshot becomes stale.

Class relationships and collaboration

Class	Important relationship or behaviour
User	Identity and UserRole value
Asset	Registered equipment and lifecycle state
Complaint	References Asset and reporting User
WorkOrder	References Complaint and assigned User
MaintenanceRecord	Submission associated with WorkOrder
RepairDetail	Record containing complaint, work and history lists
AceOsService	Coordinates session, repositories and transactions
Transactions	Implements AutoCloseable
WorkspaceView	Extends JavaFX BorderPane

Relationship model

```
Asset <--- Complaint ---> User (reporter)
             ^
             |
         WorkOrder ------> User (technician)
             ^
             |
      MaintenanceRecord

View -> AceOsService -> Repository -> JDBC/MySQL
```

These are associations and has-a relationships. A WorkOrder referencing a Complaint does not establish that deleting the Java object should delete database history. Foreign keys and service rules preserve referenced records. Java object association and strict UML composition are therefore not interchangeable claims.

The relationship view focuses on the main repair domain. It deliberately omits low-level getters and every interface widget so that the actual lifecycle collaboration remains readable.

Encapsulation and valid state changes

WorkOrder stores status and timestamps in private fields. Callers use named operations such as start, complete, returnForCorrection, resubmitCorrection and verify. The class validates the allowed source state before changing progress.

Source excerpt: WorkOrder.start

```
public void start() {
    requireStatus(WorkOrderStatus.ASSIGNED,
        "Only an assigned work order can be started");
    status = WorkOrderStatus.IN_PROGRESS;
    startedAt = LocalDateTime.now().withNano(0);
}
```

A start operation carries meaning that a generic setStatus method would not convey. It also records the first start timestamp as part of the transition. complete similarly requires IN_PROGRESS and records completion.

Encapsulation boundaries

Private fields alone do not authorize a database write. The service additionally checks the active account, role, ownership, persistent expected state and audit requirements. A model object is useful for domain invariants, but it is not a replacement for the application transaction boundary.

The implemented models vary in mutability and reconstruction needs. Record projections can contain references to mutable model objects. The project therefore demonstrates encapsulation without claiming that all entities are deeply immutable.

Source: src/main/java/com/aceos/model/WorkOrder.java.

Constructors and method overloading

Overloading gives a class more than one callable signature under the same name. AceOS uses this for creating a new identity and reconstructing an existing identity from stored data. Constructor delegation keeps common assignment in one place.

Source excerpt: User constructors

```
public User(String name, String email, UserRole role) {
    this(0, name, email, role);
}

public User(int userId, String name, String email,
           UserRole role) {
    this.userId = userId;
    this.name = name;
    this.email = email;
    this.role = role;
}
```

The three-argument form supplies the unsaved identifier value through the four-argument form. The reconstructed form preserves a database identity. This is compile-time selection by argument list, not runtime overriding.

Lifecycle reconstruction

WorkOrder has constructors for a new assignment and fuller constructors for stored status, timestamps and submission identifiers. Reconstruction must represent an existing historical state without replaying every user action. New-assignment constructors validate the technician role and accepted complaint.

RepairDetail also provides compatibility constructors that supply empty history lists. Overloads simplify callers, while guarded services prevent compatibility entry points from becoming a way to impersonate a different actor.

Sources: model/User.java, model/WorkOrder.java and model/RepairDetail.java.

Abstraction and interface contracts

Abstraction exposes a useful contract while hiding the details that callers do not need. Views call AceOsService use cases; SQL and result-set reconstruction remain inside repositories. This is a responsibility boundary even where the service is a concrete class.

Implemented interfaces

Interface	Implementation role
java.sql.Connection	Driver/proxy connection used by repositories
PreparedStatement	Bound SQL execution without driver coupling
ResultSet	Row access using a standard JDBC contract
AutoCloseable	Deterministic transaction cleanup
Callable<T>	Typed background operation
Consumer<T>	Success callback consuming the result

Transactions implements AutoCloseable so try-with-resources invokes cleanup even when an operation fails. JDBC driver instances implement standard interfaces, allowing callers to work with the same contract rather than a driver-specific class name.

Application contract

The service's private ServiceOperation<T> interface describes an authorized operation. Generics retain the return type through that execution path. FxTasks accepts behaviour through Callable and callback interfaces, separating the work from how success or failure is displayed.

The project does not claim that every abstraction is a Java abstract class. Interfaces, named service methods and repository boundaries provide different forms of abstraction in the actual code.

Inheritance in the JavaFX interface

AceOS uses inheritance where JavaFX provides reusable layout behaviour. WorkspaceView extends BorderPane, RepairFlowView extends ScrollPane and DashboardView extends VBox. Each subtype specializes a familiar layout container with application content and event handling.

```
WorkspaceView extends BorderPane
RepairFlowView extends ScrollPane
DashboardView extends VBox
```

Inherited and specialized behaviour

BorderPane supplies top, bottom, left, right and center regions. WorkspaceView arranges navigation and the current workspace within that inherited layout. ScrollPane supplies viewport and scrolling behaviour for a long repair form. VBox supplies vertical layout for dashboard content.

Role modelling decision

Requester, technician and administrator are not subclasses. One User class holds a UserRole enum. Services branch on authorized role and current session, and views expose the corresponding permitted actions. This matches a domain in which a stored identity may change role while historical references remain valid.

Inheritance versus association

A WorkOrder has a technician; it is not a technician. A Complaint concerns an Asset; it is not an Asset subtype. Keeping those relationships as references avoids misleading inheritance trees. The implementation uses inheritance for is-a UI relationships and association for domain links.

Sources: ui/WorkspaceView.java, ui/view/RepairFlowView.java, ui/view/DashboardView.java and model/UserRole.java.

Polymorphism and method overriding

Runtime polymorphism allows a caller to invoke a contract while the concrete implementation determines the behaviour. Repositories use Connection, PreparedStatement and ResultSet references whose objects are supplied by the JDBC driver. Transactions also creates a Connection proxy with controlled borrowing semantics.

Transaction connection proxy

The proxy implements Connection and forwards ordinary methods to the owner. It suppresses close, commit and setAutoCommit calls from borrowing repositories. Code typed to Connection can therefore participate in one service transaction without knowing the concrete proxy class.

Controlled overriding in tests

The UI checks use preview services extending AceOsService and overriding selected operations. A screen can receive controlled success or failure responses to exercise recovery and presentation behaviour. This demonstrates overriding in test substitution; it is not a new production role hierarchy.

Mechanism	Example	Selection
Overloading	User constructors	Compile time; argument signature
Overriding	Preview service operations	Runtime; subclass implementation
Interface polymorphism	JDBC Connection	Runtime; implementing object

These examples distinguish the academic concepts without inventing domain subclasses merely to display inheritance. The report attributes test-only overriding to the test code and keeps it separate from runtime application behaviour.

Sources: database/Transactions.java and src/test/java/com/aceos/UiSmokeTest.java.

Associations, records and collection safety

The core object graph links a complaint to its asset and reporter, and a work order to its complaint and assigned technician. Repositories reconstruct those objects from joined relational facts. Those references give the service and view meaningful objects rather than unrelated integer identifiers alone.

Source excerpt: defensive list copies

```
public RepairDetail {
    timeline = List.copyOf(timeline);
    notes = List.copyOf(notes);
    submissions = List.copyOf(submissions);
}
```

RepairDetail is a record projection for an authorized detail view. Copying its lists prevents a caller from structurally modifying those list instances after construction. It does not automatically clone the Complaint, WorkOrder or every element inside those lists.

Bounded projections

RepairSubmission, TechnicianWorkload and AssetProfile package data needed by particular views. This avoids exposing an unrestricted database session to the interface. The service decides which projection is permitted for a requester, technician or administrator.

Identity and history

Object references express current relationships; persisted primary and foreign keys preserve durable identity between sessions. Immutable repair-submission rows and append-only reviews retain earlier work even when the current work-order status changes. Domain objects and relational history serve complementary responsibilities.

Sources: model/RepairDetail.java, model/RepairSubmission.java and docs/oop-design.md.

Enums, generics and type safety

Enums constrain the vocabulary available to Java callers. UserRole identifies REQUESTER, TECHNICIAN and ADMINISTRATOR. WorkOrderStatus defines ASSIGNED, IN_PROGRESS, COMPLETED, RETURNED and VERIFIED. ComplaintStatus expresses a separate complaint lifecycle.

Separate types for separate meanings

A completed repair is not automatically a resolved complaint, and an asset's manual lifecycle is not identical to effective availability. Distinct enums reduce accidental mixing of these meanings. SQL constraints protect persisted strings as a second boundary.

Generic asynchronous contract

```
public static <T> void run(
    Callable<T> operation,
    Consumer<T> success,
    Consumer<AceOsServiceException> failure,
    Runnable finished)
```

The operation returns T and the success callback accepts T. This preserves the relationship between the producer and consumer without unchecked casts at every call site. A completion action runs regardless of success or failure, allowing the interface to clear its saving state.

Limits of static typing

A well-typed WorkOrder can still be stale or belong to another technician. Type safety cannot determine whether a session was disabled after the object loaded or whether a newer correction exists. Persistent service validation remains necessary alongside Java types.

Sources: model enums and ui/FxTasks.java.

Exceptions and resource management

AceOS distinguishes invalid domain operations from persistence failures. Models use IllegalArgumentException for invalid inputs and IllegalStateException for invalid transitions. Services translate failures into AceOsServiceException so views receive an application-level failure contract.

Transaction cleanup excerpt

```
@Override
public void close() throws SQLException {
    CURRENT.remove();
    try {
        if (!committed) owner.rollback();
    } finally {
        owner.close();
    }
}
```

Transactions is AutoCloseable. If an operation exits without a successful commit, close attempts rollback and then closes the owner even if rollback fails. Removing the thread-local borrowed connection prevents it from leaking into later operations on the same thread.

User-facing failure behaviour

The interface retains failed drafts, prevents duplicate saves and distinguishes a successful write from a failed follow-up read. Retrying a refresh should not repeat the save. This translates exception handling into a practical interaction rule rather than merely printing a stack trace.

The close excerpt is from database/Transactions.java. Service and FxTasks exception handling preserve the underlying cause for diagnostics while returning a useful operation-level message to the interface.

Repair lifecycle as object behaviour

The repair domain has explicit transitions. Assignment allows a technician to begin. Completion creates evidence for administrator review. Return requests a correction while retaining the assigned technician. Verification accepts an exact current submission and resolves the complaint.

```
ASSIGNED -> IN_PROGRESS -> COMPLETED -> VERIFIED
                            | ^
                            v |
                          RETURNED
```

Correction invariants

returnForCorrection requires an administrator, COMPLETED status, no verified timestamp and valid start/completion evidence. resubmitCorrection requires the same assigned technician and RETURNED status. It changes progress back to COMPLETED while preserving the first completion time.

Object and database cooperation

The model represents the valid transition, but repository writes preserve the numbered submission and terminal review history. Expected submission and return-review identifiers prevent a stale object from applying to a newer correction cycle. The service performs these checks within the same transaction.

Verified repairs cannot return. Reassignment is restricted to work that has not initially started. These rules avoid rewriting accepted evidence or obscuring who completed the correction. Complaint resolution closes only the associated complaint; other asset restrictions can remain.

Sources: model/WorkOrder.java, repository/RepairReviewRepository.java and service/AceOsService.java.

Authentication and guarded service actions

AceOsService retains the active Session and checks its validity before exposing the signed-in user. Authentication verifies credentials and creates the session; account changes increment authentication versions so a stale session cannot continue writing after invalidation.

Operation boundary

```
View action
  -> validate session and required role
  -> enter authorized SQL scope
  -> check ownership and current evidence
  -> perform domain/repository operation
  -> append audit and commit
```

The actor is taken from the authenticated session rather than accepted as an arbitrary user identifier from a screen. Compatibility overloads reject mismatched actor IDs. Scope rules constrain which histories and counts each role can read.

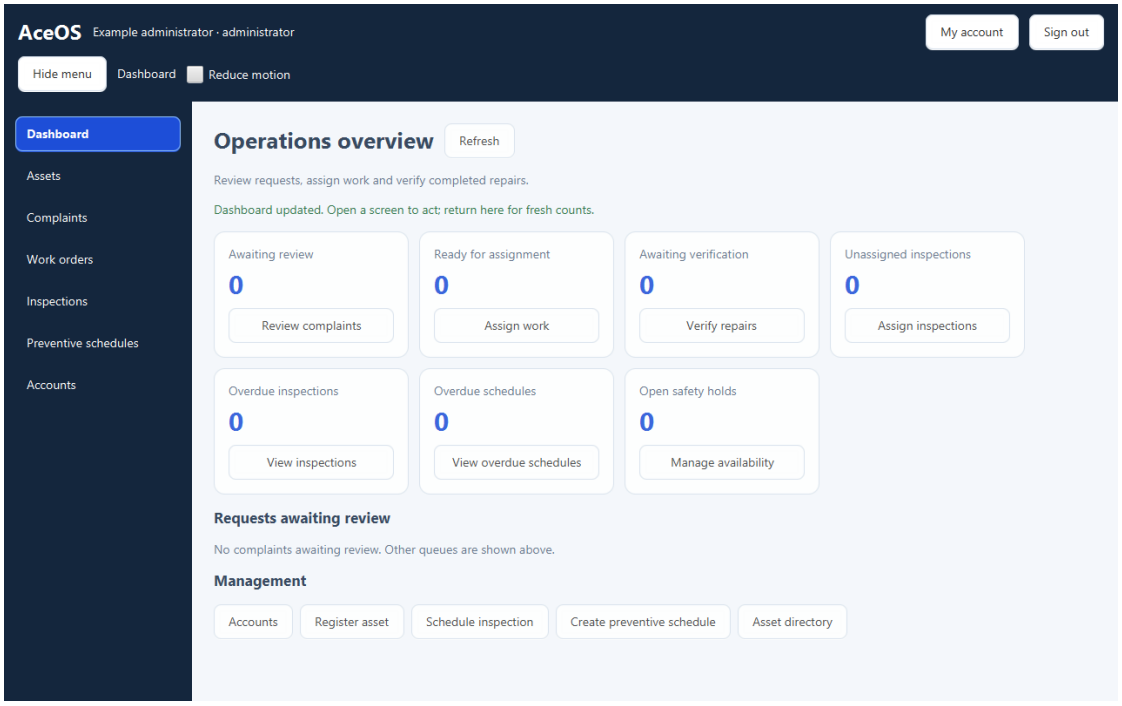
Responsibility and limitation

Models can validate a supplied role, but only the service can establish the currently authenticated actor and coordinate persistent authorization. This separation prevents a simple model method from being mistaken for a complete security boundary.

The deployment remains a trusted local client with database access. A modified client or leaked database credentials can bypass Java checks. Remote multi-user authorization is future work and is not claimed in this submission.

Sources: service/AceOsService.java, security/Authentication.java, Session.java and Scope.java.

JavaFX interaction and background work



Controlled administrator workspace with seven navigation destinations.

JavaFX views expose role-appropriate workspaces and run database operations through FxTasks. The helper starts a virtual thread for the operation and posts success or failure callbacks through Platform.runLater so interface updates happen on the JavaFX application thread.

Interaction safeguards

Saving blocks duplicate action and re-entry. Unfinished-form protection defaults to staying on the page. Failed drafts remain available, and an operation that saved successfully is not silently repeated when its follow-up read fails. Labels, focus paths and visible actions were included in the recorded UI verification.

The screenshot is a controlled verification fixture from target/ui-checks. It demonstrates the implemented workspace layout; it is not a screenshot of private production records.

Sources: ui/FxTasks.java, ui/NavigationGuard.java and docs/verification.md.

Verification and implementation results

The recorded final v2.0.0 verification run passed 89 checks: 46 database checks, 3 password/session checks and 40 JavaFX checks, with zero failures, errors or skips. These figures come from the project verification documentation; the application test suite was not rerun while preparing this report.

Area	Recorded evidence
Relational behaviour	Rollback, stale evidence, competing actions, foreign keys
Authorization	Role, ownership and invalidated-session checks
Repair history	Return, correction, exact latest-submission verification
Interface	Draft retention, duplicate-save guards and recovery
Human walkthrough	Five repair-return steps confirmed by the project author

The focused repair-return checks overlap the final suite and must not be added to 89 as extra coverage. UI snapshots are controlled verification fixtures, not production usage statistics. Layout review was recorded at 800 x 520 and 1280 x 800.

Interpretation

The evidence supports the implemented workflows within the configured local environment. It does not establish production scalability, remote security enforcement or performance under large institutional workloads. Database tests require the disposable test schema; a skipped database test is not evidence of correctness.

OOP-specific evidence

Model transitions are exercised through application/database scenarios. UI preview services demonstrate overriding through controlled substitution. Code inspection links each academic concept to a named implementation rather than treating a successful compilation as proof of all object-oriented design claims.

Limitations, future scope and conclusion

AceOS meets the closed college scope for local asset and maintenance operations. It preserves repair history, prevents stale verification and keeps independent safety restrictions authoritative. The main design result is that complaint closure, repair acceptance and asset availability remain distinct decisions.

Current limitations

The trusted local Java client is the authorization boundary. A modified client or someone holding database credentials can bypass Java checks. The system does not provide a hosted backend, remote enforcement, cancellation of verified repairs, reassignment after repair start or automatic clearance of safety holds. Preventive completion advances the next due date without a separate completion-history table.

Future scope

Possible extensions include notifications, QR-based asset lookup, attachments, calendar integration and richer reporting. A future shared deployment would need server-side authorization and an explicit concurrency and operational design. These are proposed extensions, not features claimed in the submitted application.

Conclusion

The project demonstrates how clear responsibilities and explicit evidence links improve correctness. Its most instructive case is a returned repair: the technician adds a correction, the administrator accepts one exact submission, and the system preserves the earlier submissions and reviews for later explanation.

References and source evidence

Primary evidence is the AceOS repository and its existing documentation. Paths below are repository-relative. The report describes current v2.0.0 behaviour; the v2.0.0 release tag identifies the packaged source.

Reference	Purpose
README.md; docs/college-scope.md	Application scope and release context
docs/data-dictionary.md	All 16 tables, fields and key rules
docs/er-diagram.md	Complete foreign-key relationship map
docs/transactions.md	Atomic operations and consistency policy
docs/oop-design.md	Implemented OOP concept mapping
docs/verification.md	Recorded verification and limitations
docs/foundation.md; docs/run-guide.md	Setup, migration and recovery
pom.xml	Build dependencies and application version
src/main/resources/database/V000-V005	Baseline and versioned schema resources
src/main/java/com/aceos/	Application implementation
src/test/java/com/aceos/	Database, security and UI checks

Repository: https://github.com/SmitHitnalli/aceos

Evidence conventions

SQL examples marked illustrative explain the schema and are not measured query results. Java excerpts are drawn from named source files; simplified diagrams describe implemented relationships and are not claims of additional runtime classes. Screenshots are copied from the controlled UI-check evidence. No passwords, connection secrets or private database rows are included.